

Week

5

INTRO TO Git:

- Concept of Version Control System
- Concept of Git
- Some Basic Commands



VERSION CONTROL

Most used Concurrent Version System (CVS) and Subversion (SVN).

Key Concepts:

- Central Source Control System
- Users Check Code into Server
- Requires connection to the server to perform commits, updates etc.

WHAT IS GIT?



GIT

Git is a popular version control system. It was created by Linus Torvalds in 2005, and has been maintained by Junio Hamano since then.

It is used for:

- Tracking code changes
- Tracking who made changes
- Coding collaboration



WHAT DOES GIT DO?

- Manage projects with **Repositories**
- **Clone** a project to work on a local copy
- Control and track changes with **Staging** and **Committing**
- **Branch** and **Merge** to allow for work on different parts and versions of a project
- **Pull** the latest version of the project to a local copy
- **Push** local updates to the main project



WORKING WITH GIT

- Initialize Git on a folder, making it a **Repository**
- Git now creates a hidden folder to keep track of changes in that folder
- When a file is changed, added or deleted, it is considered **modified**
- You select the modified files you want to **Stage**
- The **Staged** files are **Committed**, which prompts Git to store a **permanent** snapshot of the files
- Git allows you to see the full history of every commit.
- You can revert back to any previous commit.
- Git does not store a separate copy of every file in every commit, but keeps track of changes made in each commit!



WHY GIT?

- Over 70% of developers use Git!
- Developers can work together from anywhere in the world.
- Developers can see the full history of the project.
- Developers can revert to earlier versions of a project.

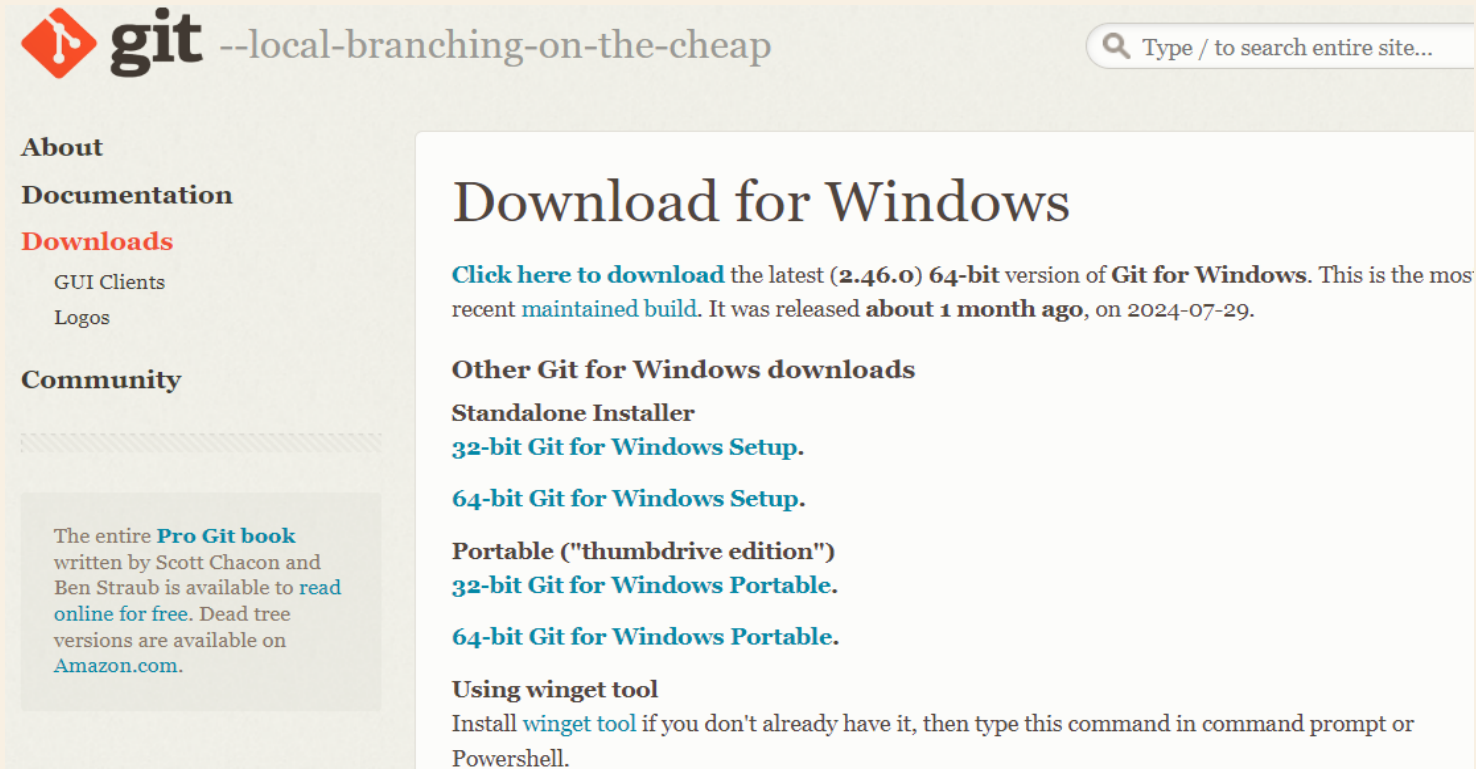


GIT GETTING STARTED



GIT INSTALL

You can download Git for free from the following website: <https://www.git-scm.com/>



The screenshot shows the Git website homepage. At the top, there is a navigation bar with the Git logo (an orange diamond with a white branching diagram) and the text "git --local-branching-on-the-cheap". To the right of the logo is a search bar with the placeholder text "Type / to search entire site...". Below the navigation bar, there is a sidebar on the left with links to "About", "Documentation", "Downloads", "GUI Clients", "Logos", and "Community". The main content area is titled "Download for Windows" and contains the following text: "Click here to download the latest (2.46.0) 64-bit version of Git for Windows. This is the most recent maintained build. It was released about 1 month ago, on 2024-07-29." Below this, there are sections for "Other Git for Windows downloads" which include "Standalone Installer", "32-bit Git for Windows Setup.", "64-bit Git for Windows Setup.", "Portable ('thumbdrive edition')", "32-bit Git for Windows Portable.", and "64-bit Git for Windows Portable.". At the bottom, there is a section titled "Using winget tool" which says "Install winget tool if you don't already have it, then type this command in command prompt or Powershell."

git --local-branching-on-the-cheap

Type / to search entire site...

About

Documentation

Downloads

- GUI Clients
- Logos

Community

The entire **Pro Git book** written by Scott Chacon and Ben Straub is available to read [online for free](#). Dead tree versions are available on [Amazon.com](#).

Download for Windows

Click here to download the latest (2.46.0) 64-bit version of **Git for Windows**. This is the most recent **maintained build**. It was released **about 1 month ago**, on 2024-07-29.

Other Git for Windows downloads

Standalone Installer

- 32-bit Git for Windows Setup.**
- 64-bit Git for Windows Setup.**

Portable ("thumbdrive edition")

- 32-bit Git for Windows Portable.**
- 64-bit Git for Windows Portable.**

Using winget tool

Install **winget tool** if you don't already have it, then type this command in command prompt or Powershell.



WORK FLOW



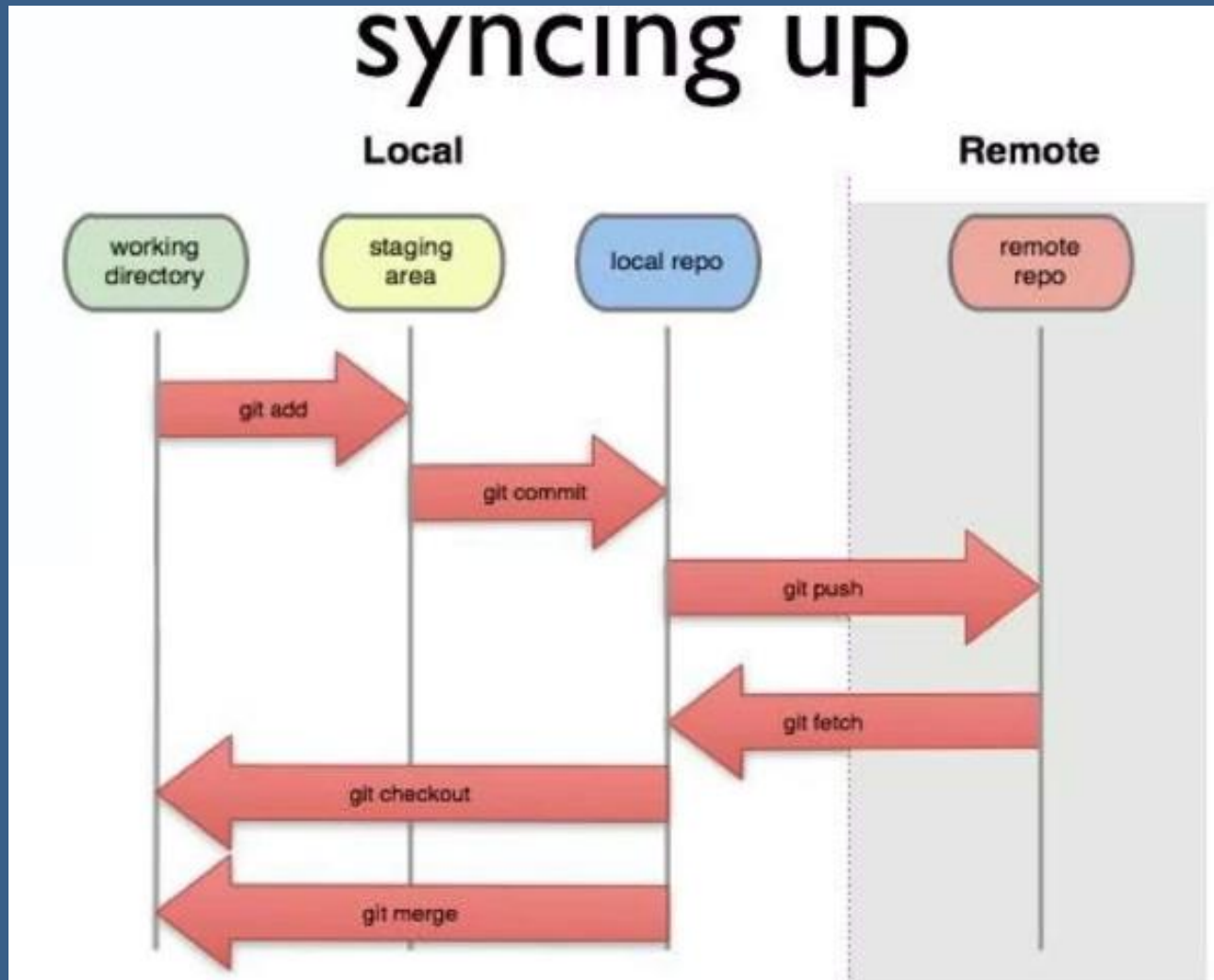
USEFUL BASIC COMMANDS

git init	▸ repository setup
git add	▸ add files to queue for next commit
git commit	▸ commit queued files
git push	▸ push commit(s) to remote repository
git pull	▸ fetch changes from remote repository
git clone	▸ clone repository into a local directory
.gitignore	▸ ignore specific files by adding them here

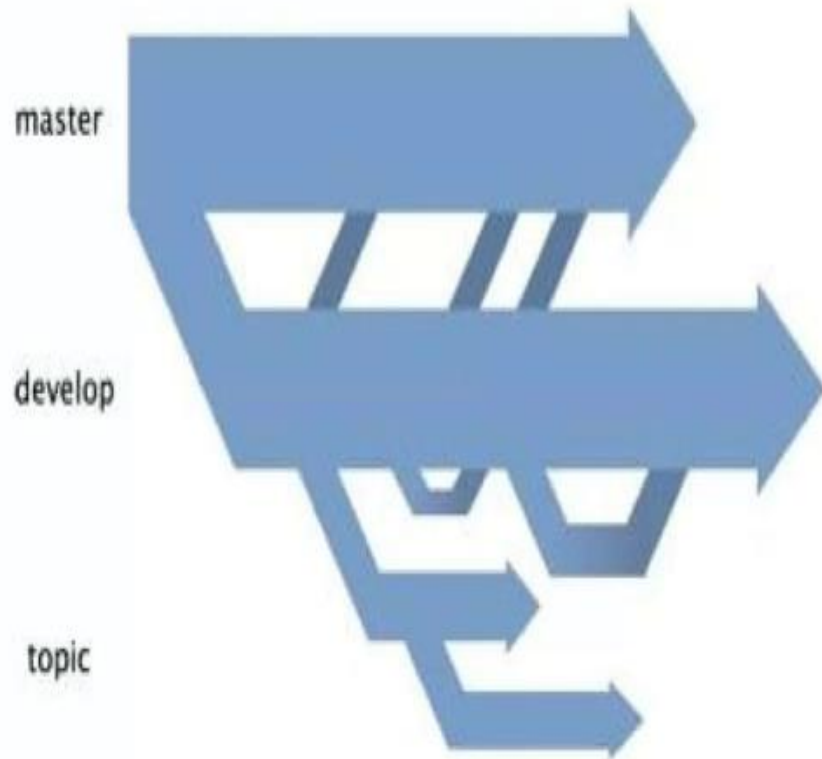
USEFUL BASIC COMMANDS

<code>git status</code>	▸ Show uncommitted changes
<code>git rm</code>	▸ Remove a file from a repository
<code>git mv</code>	▸ Move a file within repository
<code>git diff</code>	▸ Generate a differences between multiple commits
<code>git log</code>	▸ View a log of commits

GIT LEVEL



BRANCHES



- Cheap and local
- Easy to switch
- Try out new ideas
- Merging is way smarter

BRANCHING

<code>git branch new_branch</code>	create new branch
<code>git branch -b feature2</code>	create new branch and switch to that branch
<code>git commit</code>	save some work
<code>git checkout master</code>	switch back
<code>git merge feature2</code>	work is merged in
<code>git rebase feature2</code>	work played on top
<code>git branch -d feature2</code>	delete branch

TAGGING AND MERGING

<code>git merge branch_name</code>	Merge changes from <code>branch_name</code> to the current branch
<code>git tag -a name</code>	Add a tag with a indicated name
<code>git tag -l</code>	List tags
<code>git push --tags</code>	Push tags to remote repository

HANDLING SOME CRITICAL SITUATION

- Changing your last commit
git commit --amend
- Unstaging a staged File
git reset HEAD FILE_NAME
- Unmodifying a modified File
git checkout -- FILE_NAME

CREATING GIT FOLDER

mkdir **makes** a **new directory**.

cd **changes** the **current working directory**.

```
chris@cmmansueto MINGW64 ~  
$ mkdir myDemoProject
```

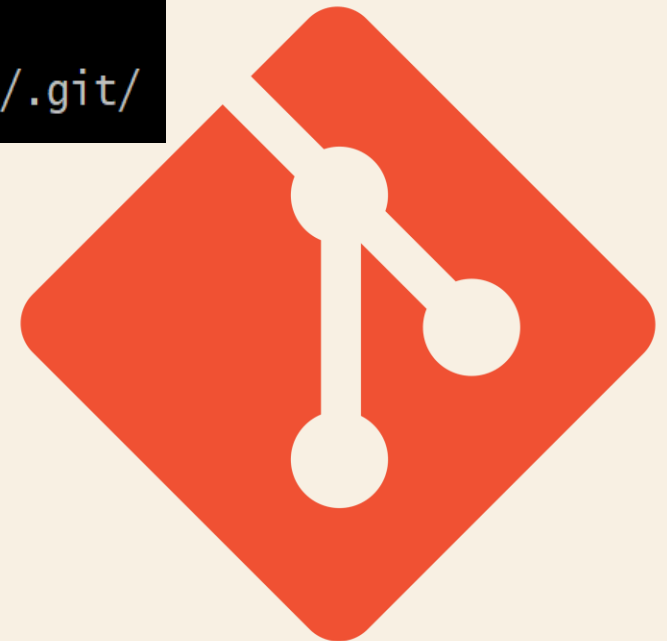
```
chris@cmmansueto MINGW64 ~  
$ cd myDemoProject
```



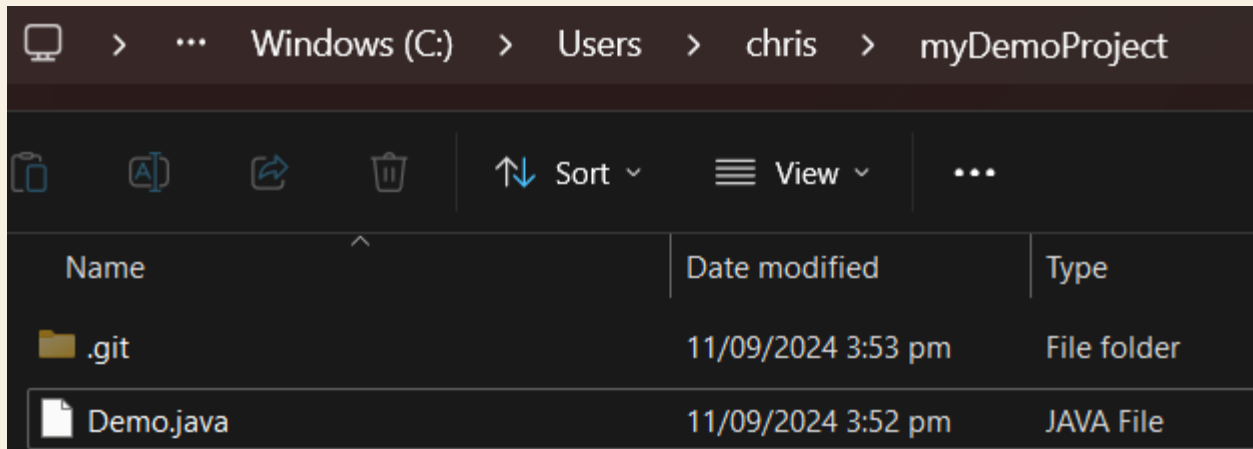
INITIALIZE GIT

Once you have navigated to the correct folder, you can initialize Git on that folder:

```
chris@cmmansueto MINGW64 ~/myDemoProject  
$ git init  
Initialized empty Git repository in C:/Users/chris/myDemoProject/.git/
```



ADD A NEW FILES ON LOCAL GIT REPO



A screenshot of a Windows File Explorer window showing the contents of a local Git repository. The address bar indicates the path: Windows (C:) > Users > chris > myDemoProject. The file list shows two items: a folder named .git and a file named Demo.java. The .git folder was modified on 11/09/2024 at 3:53 pm and is a File folder. The Demo.java file was modified on 11/09/2024 at 3:52 pm and is a JAVA File.

Name	Date modified	Type
.git	11/09/2024 3:53 pm	File folder
Demo.java	11/09/2024 3:52 pm	JAVA File

```
chris@cmmansueto MINGW64 ~/myDemoProject (master)
$ ls
Demo.java
```

ls will **list** the files in the directory.



GIT STATUS

Now Git is **aware** of the file, but has not **added** it to our repository!

Files in your Git repository folder can be in one of 2 states:

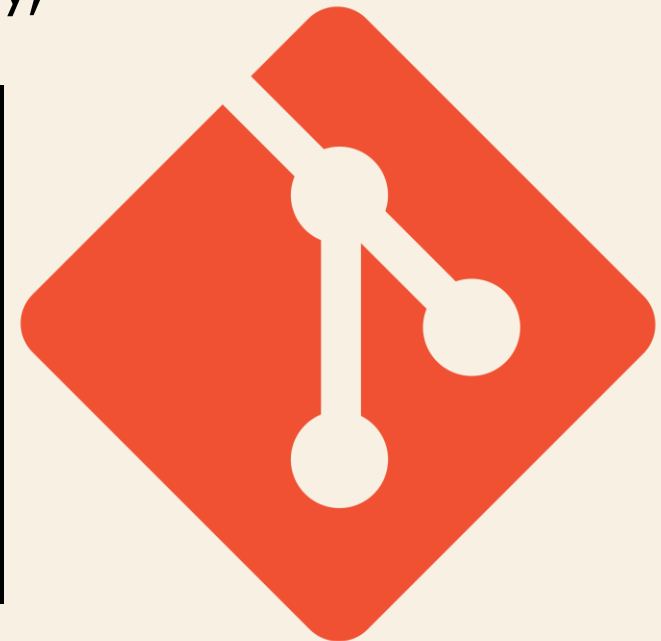
- Tracked - files that Git knows about and are added to the repository
- Untracked - files that are in your working directory, but not added to the repository

```
chris@cmmansueto MINGW64 ~/myDemoProject (master)
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    Demo.java

nothing added to commit but untracked files present (use "git add" to track)
```



GIT STAGING ENVIRONMENT

One of the core functions of Git is the concepts of the Staging Environment, and the Commit.

As you are working, you may be adding, editing and removing files. But whenever you hit a milestone or finish a part of the work, you should add the files to a Staging Environment.

Staged files are files that are ready to be **committed** to the repository you are working on.

```
chris@cmmansueto MINGW64 ~/myDemoProject (master)
$ git add Demo.java

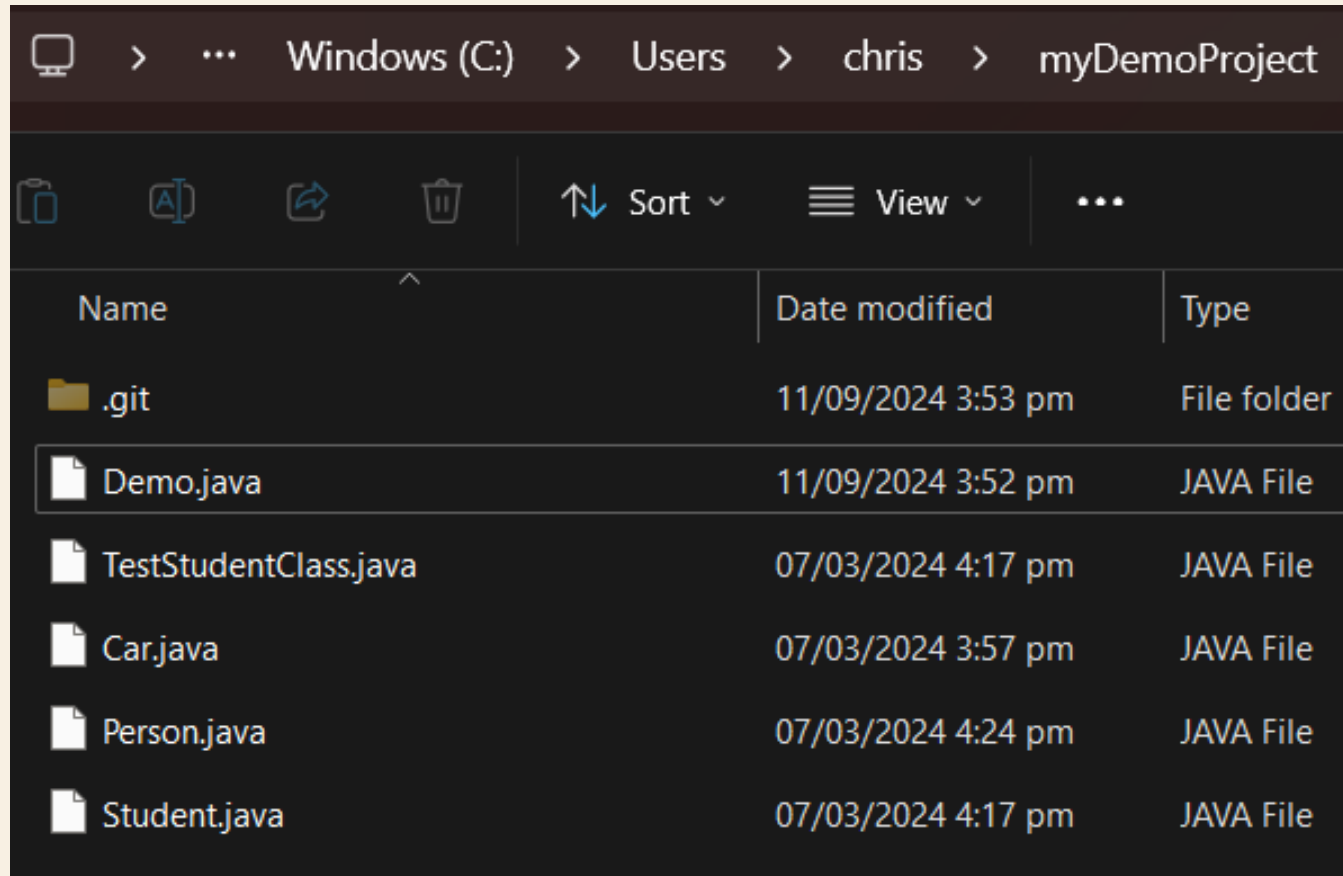
chris@cmmansueto MINGW64 ~/myDemoProject (master)
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   Demo.java
```



GIT STAGING ENVIRONMENT



The screenshot shows a Windows File Explorer window with the address bar displaying the path: Windows (C:) > Users > chris > myDemoProject. The toolbar includes icons for file operations and a 'Sort' dropdown menu. The main area displays a list of files and folders with columns for Name, Date modified, and Type.

Name	Date modified	Type
.git	11/09/2024 3:53 pm	File folder
Demo.java	11/09/2024 3:52 pm	JAVA File
TestStudentClass.java	07/03/2024 4:17 pm	JAVA File
Car.java	07/03/2024 3:57 pm	JAVA File
Person.java	07/03/2024 4:24 pm	JAVA File
Student.java	07/03/2024 4:17 pm	JAVA File



GIT STAGING ENVIRONMENT

```
chris@cmmansueto MINGW64 ~/myDemoProject (master)
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   Demo.java

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        Car.java
        Person.java
        Student.java
        TestStudentClass.java
```



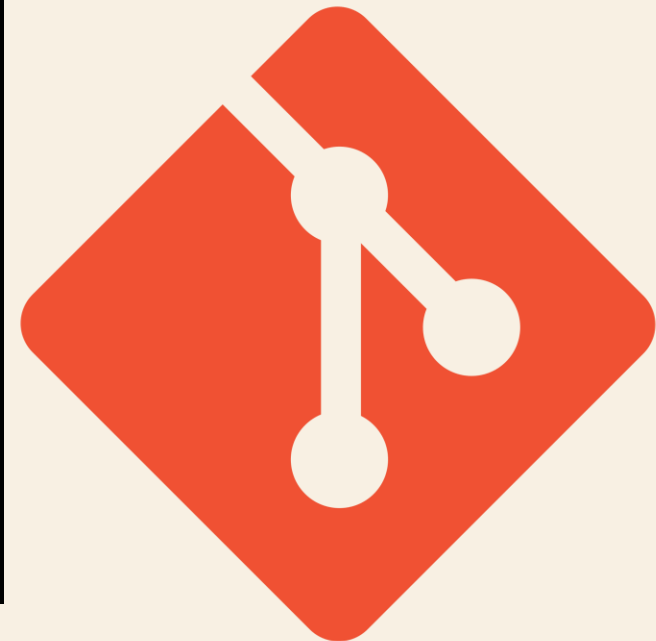
GIT ADD MORE THAN ONE FILE

```
chris@cmmansueto MINGW64 ~/myDemoProject (master)
$ git add --all

chris@cmmansueto MINGW64 ~/myDemoProject (master)
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   Car.java
    new file:   Demo.java
    new file:   Person.java
    new file:   Student.java
    new file:   TestStudentClass.java
```



GIT STAGING ENVIRONMENT

One of the core functions of Git is the concepts of the Staging Environment, and the Commit.

As you are working, you may be adding, editing and removing files. But whenever you hit a milestone or finish a part of the work, you should add the files to a Staging Environment.

Staged files are files that are ready to be **committed** to the repository you are working on.

```
chris@cmmansueto MINGW64 ~/myDemoProject (master)
$ git add Demo.java

chris@cmmansueto MINGW64 ~/myDemoProject (master)
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   Demo.java
```

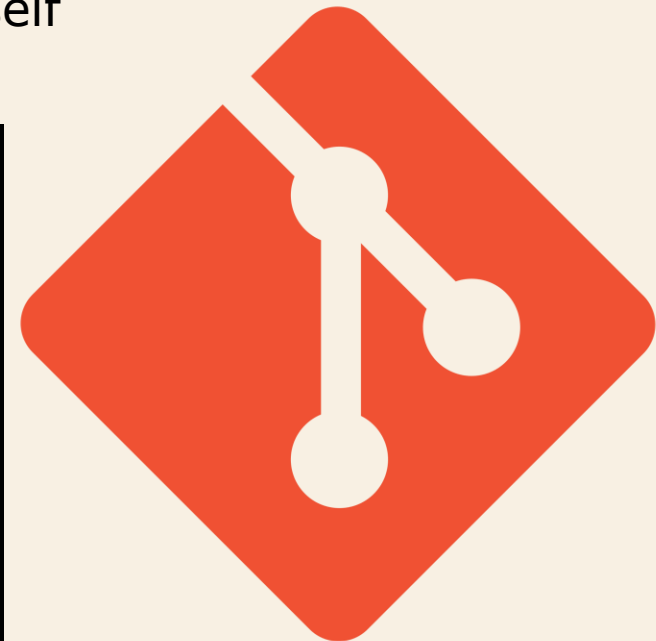


GIT COMMIT

Adding commits keep track of our progress and changes as we work. Git considers each commit change point or "save point". It is a point in the project you can go back to if you find a bug, or want to make a change.

When we commit, we should **always** include a **message**. By adding clear messages to each commit, it is easy for yourself (and others) to see what has changed and when.

```
chris@cmmansueto MINGW64 ~/myDemoProject (master)
$ git commit -m "My Demo Project for JAVA OOP"
[master (root-commit) 5b2dc67] My Demo Project for JAVA OOP
5 files changed, 195 insertions(+)
create mode 100644 Car.java
create mode 100644 Demo.java
create mode 100644 Person.java
create mode 100644 Student.java
create mode 100644 TestStudentClass.java
```



GIT COMMIT

Updating the Demo.java file.

```
chris@cmmansueto MINGW64 ~/myDemoProject (master)
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   Demo.java
```

```
chris@cmmansueto MINGW64 ~/myDemoProject (master)
$ git add Demo.java

chris@cmmansueto MINGW64 ~/myDemoProject (master)
$ git commit -m "Updated Demo.java"
[master 606bac7] Updated Demo.java
1 file changed, 1 insertion(+), 1 deletion(-)
```



GIT COMMIT LOG

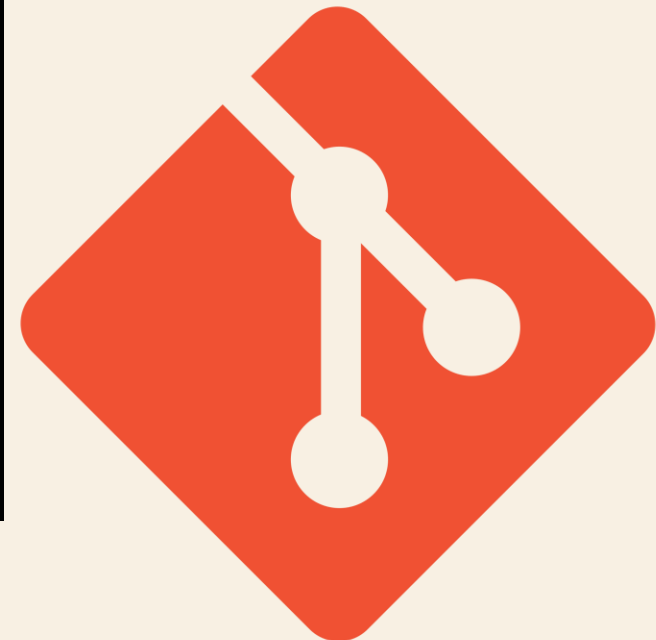
To view the history of commits for a repository, you can use the log command:

```
chris@cmmansueto MINGW64 ~/myDemoProject (master)
$ git log
commit 606bac7f4480232cf38ee4f4c31c1917ed4b4bdc (HEAD -> master)
Author: cmmansueto <christianmichael.mansueto@umak.edu.ph>
Date:   Wed Sep 11 15:53:57 2024 +0800

    Updated Demo.java

commit 5b2dc675f538e36d53db0481cf6b4ca0f0bee3a7
Author: cmmansueto <christianmichael.mansueto@umak.edu.ph>
Date:   Wed Sep 11 15:51:29 2024 +0800

    My Demo Project for JAVA OOP
```



WORKING WITH GIT BRANCHES

In Git, a branch is a new/separate version of the main repository.

Let's say you have a large project, and you need to update the design on it.

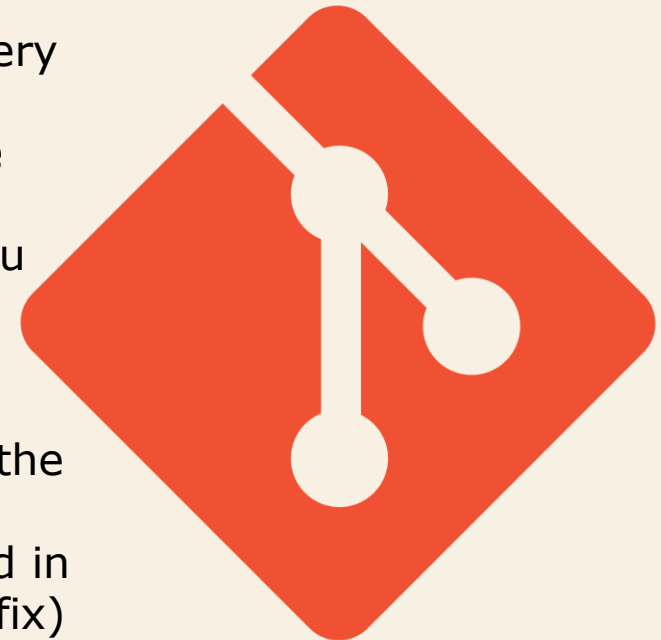


WORKING WITH GIT BRANCHES

How would that work without and with Git:

Without Git:

- Make copies of all the relevant files to avoid impacting the live version
- Start working with the design and find that code depend on code in other files, that also need to be changed!
- Make copies of the dependant files as well. Making sure that every file dependency references the correct file name
- EMERGENCY! There is an unrelated error somewhere else in the project that needs to be fixed ASAP!
- Save all your files, making a note of the names of the copies you were working on
- Work on the unrelated error and update the code to fix it
- Go back to the design, and finish the work there
- Copy the code or rename the files, so the updated design is on the live version
- (2 weeks later, you realize that the unrelated error was not fixed in the new design version because you copied the files before the fix)



WORKING WITH GIT BRANCHES

With Git:

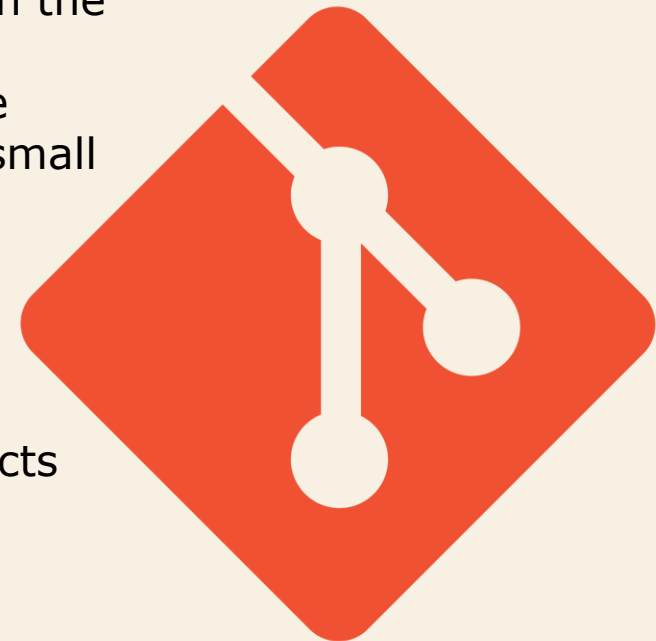
- With a new branch called new-design, edit the code directly without impacting the main branch
- EMERGENCY! There is an unrelated error somewhere else in the project that needs to be fixed ASAP!
- Create a new branch from the main project called small-error-fix
- Fix the unrelated error and merge the small-error-fix branch with the main branch
- You go back to the new-design branch, and finish the work there
- Merge the new-design branch with main (getting alerted to the small error fix that you were missing)

Branches allow you to work on different parts of a project without impacting the main branch.

When the work is complete, a branch can be merged with the main project.

You can even switch between branches and work on different projects without them interfering with each other.

Branching in Git is very lightweight and fast!



NEW GIT BRANCH

```
chris@cmmansueto MINGW64 ~/myDemoProject (master)
$ git branch myDemoBranch
```

```
chris@cmmansueto MINGW64 ~/myDemoProject (master)
$ git checkout myDemoBranch
Switched to branch 'myDemoBranch'
D      Car.java
D      Person.java
D      Student.java
D      TestStudentClass.java

chris@cmmansueto MINGW64 ~/myDemoProject (myDemoBranch)
$ ls
Demo.java
```



NEW GIT BRANCH

```
chris@cmmansueto MINGW64 ~/myDemoProject (master)
$ git branch
* master
  myDemoBranch
```

```
chris@cmmansueto MINGW64 ~/myDemoProject (master)
$ git checkout myDemoBranch
Switched to branch 'myDemoBranch'
```



MERGE BRANCHES

```
chris@cmmansueto MINGW64 ~/myDemoProject (master)
$ git merge myDemoBranch
Updating 606bac7..7f146da
Fast-forward
 Car.java          | 54 -----
 Demo.java         |  2 +-
 Person.java       | 41 -----
 Student.java      | 60 -----
 TestStudentClass.java | 16 -----
5 files changed, 1 insertion(+), 172 deletions(-)
delete mode 100644 Car.java
delete mode 100644 Person.java
delete mode 100644 Student.java
delete mode 100644 TestStudentClass.java

chris@cmmansueto MINGW64 ~/myDemoProject (master)
$ ls
Demo.java
```



An abstract geometric design on the left side of the slide. It features a dark blue background with various geometric shapes and patterns. A white circle is positioned near the top left. Below it, a light blue semi-circle is visible. To the right of the semi-circle, there is a pink triangle with diagonal lines. Below the semi-circle, there is a pink square with a pattern of concentric lines. To the right of the square, there is a light blue triangle. Below the square, there is a pink triangle. To the right of the triangle, there is a dark blue triangle. The overall design is modern and minimalist.

THANK YOU